Problem Set 1: Getting Started

6.4210/2 Robotic Manipulation — Fall 2026

Due: September 17, 2026

Out: September 10 **Due:** September 17

Relevant reading in the course notes:

- Ch. 1 — Introduction
- Ch. 2 — Let's get you a robot

The homework assignments in this class have the primary objective of *teaching*, not assessment. The real assessments are the quizzes and the final project report and video. Use these assignments to help focus your study and to help you interrogate your own understanding.

In general, each "bookwork" question comes in two pieces: an immediately graded part, marked **Gradescope**, whose answer is a choice or a literal number or word that you enter in Gradescope; and a more open-ended part, marked **Going further**, that you should engage with to interrogate your own understanding. We will hand out solutions to these parts, and the exams will expect this depth of understanding.

The "computational questions" involve getting simulated robots to perform various tasks. You are welcome to use any tools you wish to do these questions. However, it's important that you remain engaged with and critical of the process. We will ask you to (1) provide some "checkpoint" answers to concrete questions about the results of your code in gradescope and (2) to make an upload a movie of your robot working.

After the computational questions comes a set of reflection questions indicating our expectations for what you should understand about your code, however it was produced.

1 Bookwork

Problem 1 State [3]

The *state* of a system is the minimal information about its past that is sufficient to determine its future states and outputs, given future inputs.

Gradescope: What state is needed for each of the following dynamical systems? Say what it is, and enter how many dimensions it has in total.

(a) A pendulum in the plane. Dimension: _____ [1]

(b) A free body in 3D. Dimension: _____ [1]

(c) A mobile robot in the plane that discharges its battery as it moves, and can re-fill it at certain locations. Take it to be slow enough that its velocity can be ignored. Dimension: _____ [1]

Going further: What if you bolt the pendulum's pivot onto a cart that you can drive back and forth along a horizontal rail. By how much does the state grow, and why? What if instead you leave the pivot fixed but replace the rigid rod with a spring that can stretch along its length?

Problem 2 Configuration space [3]

Gradescope: For each of these robots, give the dimension of its configuration space and the dimension of its actuation space.

(a) A two-wheeled differential-drive mobile robot. [1]
Configuration: _____ Actuation: _____

(b) A Kuka IIWA manipulator. [1]
Configuration: _____ Actuation: _____

(c) A flying drone with four fixed propellers. [1]
Configuration: _____ Actuation: _____

Going further: Here are two cases with more complicated answers.

(d) A snake.

(e) An articulated ("bendy") city bus: two rigid sections joined by a vertical hinge, with the rear section driving and the front section steering. What is the dimension of its configuration space now, and what is actuated?

Problem 3 Manipulation [1]

Gradescope: What makes robot manipulation fundamentally different from robot driving?

A. Perception is not a key issue in robot manipulation.

B. Manipulators have more degrees of freedom than mobile robots.

C. Manipulation does not involve interaction with people.

D. Robot manipulation is about changing the environment through selective contact, while driving requires always avoiding contact with the environment (beyond the wheels and road).

Going further: What about a snowplow or a robot vacuum that can nudge chairs out of its way? Are these systems more like automated cars or like manipulation robots?

Problem 4 Manipulation systems [1]

Gradescope: Which of these describe prominent approaches to the design of robot manipulation systems? (Select all that apply.)

◯ End-to-end imitation learning (pixels to motor commands)

◯ End-to-end reinforcement learning (pixels to motor commands)

◯ Sense-Plan-Act architectures

◯ Scripted sequences of motion commands

Going further: Pick two of these four approaches and think about the resources they need: a simulator, demonstrations, object models, robot models, engineering time. How do they trade off against each other?

Problem 5 Joint types [1]

Gradescope: For each of the mechanisms below, indicate what kind of joint(s) it has.

(a) Kuka IIWA main joints A. revolute B. prismatic C. fixed D. free

(b) A cupboard drawer A. revolute B. prismatic C. fixed D. free

(c) A parallel-jaw gripper A. revolute B. prismatic C. fixed D. free

(d) A block resting on the table A. revolute B. prismatic C. fixed D. free

Going further: What kind of joint is the lid of a screw-top jar, or a bolt going into a threaded hole? What about a shoulder? How many degrees of freedom does each have, and can you build either one out of the four joint types above?

Problem 6 Drake [1]

Gradescope: Drake is a large software library. What are some important components? (Select all that apply.)

◯ Block diagram specification of systems, based on the "signals and systems" abstraction

◯ Extensive optimization toolkit

- ◯ A sophisticated "physics engine" for multi-body dynamics
- ◯ GPU parallelized simulations
- ◯ Deterministic replay of simulations

Going further: Why do we care about deterministic replay of simulations?

2 Computational questions

Initial Setup and Installation

Follow the setup instructions in the README of the handouts repository:

https://github.com/tlpmit/6.4210-2-fall26-handouts

It covers installing the prerequisites (git, Python via uv, Drake, Graphviz), cloning the repository, keeping your ongoing work in a *private* GitHub repository, pulling updates when psets are released or revised, and working on Google Colab instead of a local machine.

Problem 7 Discrete-time systems in Drake [20]

Drake offers a very sophisticated set of tools for defining system blocks and wiring them up. In this lab we are going to build up a simple interface for discrete-time systems. It assumes the input, state, and output of the system can each be specified as a vector of real values, and that the operation of the system is captured by two functions:

- the **next-state function** computes $s_{t+1} = f(s_t, i_t)$, where s_t is the state at time t and i_t is the input at time t;
- the **output function** computes $o_t = g(s_t, i_t)$.

As you read the provided code and write your own functions, read the Drake C++ Documentation (we are writing in python, and there is Drake Python Documentation, but most of the C++ functions are also bound in python and the python documentation is mostly auto-generated from the C++ documentation, which is more carefully curated.) It can be helpful to add debugging print statements to understand what each line of code does. Also, use `plot_system_graphviz`, to draw a system's block diagram: each subsystem as a box listing its input and output ports, with an arrow for every connection.

(a) **A free body in 1D** Read the code for `DTVectorSystem` in `dtsystems.py`. Use it to implement a system representing a free body in 1D whose input is a force and whose output is the position of the body. Concretely, write the body of the function `Body1D` in `dtsystems.py`. [5]

Use the forward Euler integration method: $s_{t+1} = s_t + \Delta t\, \dot{s}_t$.

Use the `Constant` and `Counter` constructors that are defined in the code as examples of defining systems, and `main0` as an example of how to instantiate and simulate and add logging to a system.

Gradescope: Use `next_state_fun` on your `Body1D` to compute the state after a single update of timestep 0.1 s, for a body of mass 2 kg, from an initial position of 0.0 and a velocity of 0.5, with an input force of 0.3 N.

Position: _____ Velocity: _____

(b) **Series composition** Read the code for `series_composition`, which takes two systems and composes them by wiring the output of the first into the input of the second. Use the `Constant` constructor to make a constant force source, couple it with the body system you defined above, and use `simulate` to see where the body is after 5 steps of applying 0.1 N. Note that `simulate` takes a duration in *seconds*, not steps, so with `dt` = 0.1 you want `T` = 0.5. [5]

Use the same body as in part (a) — mass 2 kg and `dt` = 0.1 s — and start it at rest at the origin, so its initial state is (0.0, 0.0). `simulate` returns the simulator; pass that and the body to `get_state` to read out the body's final state. Use the `plot_log` helper function to plot the output of the `Body1D` system.

Gradescope: Position: _____ Velocity: _____

(c) **A position controller** Given a desired position of the body, a position controller can be used to modulate the applied force to get the body to the right position, even in the presence of some amount of disturbance. [5]

Use `DTVectorSystem` to implement a position controller with the current position as input and the force as output, where the force is proportional to the difference between the current and desired position. Write this generally, so we can use it later.

Gradescope: Use `output_fun` on your `PController` to compute the force it commands. Take a controller of dimension 1 with a set point of 1.0 and a gain of 10, and an input (the body's current position) of 0.2.

Force: _____

(d) **Feedback composition** Define `feedback_composition` which, like `series_composition`, takes two systems, but this time connects the output of each one into the input of the other. This resulting system will have no inputs or outputs. [5]

Use it to combine your controller with your body. Give it a target position of 1.0 and simulate and log the behavior of the combined system. Try some different gains.

Gradescope: Combine your `PController` with your `Body1D` using `feedback_composition`: mass 2 kg, `dt` = 0.1 s, gain 10, and a target position of 1.0. Start the body at position 0.2 with velocity 0.5, simulate for `T` = 10 s, and plot its position. Which best describes the behavior?

A. It settles at the target position of 1.0.

B. It oscillates about 1.0 forever, with constant amplitude.

C. It oscillates about 1.0 with growing amplitude.

D. It moves away from 1.0 without ever oscillating.

Problem 8 A 2D Cartesian robot [11]

Now, we'll play with a simple "robot" with 2 prismatic joints. This robot can be moved in x and y, and is rendered in Drake as a vertical cylinder on a table.

Read the code in `cartesian_2d_robot.py` to see how we define a table (no legs) as an SDF object and attach it permanently (weld it) to the world. Then we define the "finger" as a more complicated body, with inertia and damping, and two `PrismaticJoint`s.

(a) **Drive the finger with a constant input** The procedure `create_2d_robot_diagram` uses Drake tools to construct a scene with the table and the finger and put them into a physical system called a `MultibodyPlant`. [1]

Gradescope:

Run `test_const_input` with input $[4.0, 1.0]$ and interpret the plot of the state of the finger. Why do the x and y velocities plateau?

A. The input to the system is a constant velocity command.

B. There are velocity limits defined on the prismatic joints in the SDFs.

C. Friction between the bottom of the finger and the table surface grows until it balances the input force.

D. There is damping defined on the joint proportional to the velocity of the finger which balances the input force.

E. Gravity pulling down on the finger balances the input force.

(b) **Drive the finger to a set position** Modify the code to use your position controller to drive the finger to a set position (have it take the desired position as input). One problem: the plant's state output port carries $[x, y, \dot{x}, \dot{y}]$, but the controller needs just the two positions. Write the body of `Observer` in `dtsystems.py` — a stateless system whose output is the elements of its input selected by `indices` — and compose it between the plant and the controller. Like the controller, it is written generally: we will use it again on the iiwa. [5]

Gradescope: With the SDF settings already defined in the script and a position controller gain of 100, initial finger state defined by `finger_s0(FINGER_START)`, and a target position of $[0.9, 0.6]$, how long does it take for the finger to come to rest at the target position? (Use `plot_log`, and/or look at the meshcat recording)

(c) **A trajectory follower** Next, we would like to drive the finger through a series of waypoints, by defining a sequence of configurations and using the position controller to get within some epsilon of the first one, then switching to the next one, and so on. [5]

Write a `DTVectorSystem` that takes as input the current position of the robot and outputs a setpoint for the controller. It will have to use its state to keep track of which waypoint it is working on.

You will need to make a new version of your `PController` that takes the desired set-point as well as the current value as input.

Use these components to build a system that drives the robot in a `SQUARE`. Try different values of gains and epsilon.

Gradescope: Set `JOINT_DAMPING` to 20 and use a controller gain of 100. Drive the robot around `SQUARE` for `T` = 5 s. How far does the finger travel? (Use `plot_path`, which reports the length of the path it plots.)

$\epsilon = 0.1$: _____ m $\epsilon = 0.01$: _____ m

Problem 9 Simulating the IIWA [17]

Now we can apply these same ideas to a real robot. Read the code in `iiwa_1.py`. We can apply the controllers and basic strategy for doing things with the robot that we developed on the simple robot now to the real Iiwa!

(a) **Zero torque** Start the robot at configuration $[0, 1.0, 0.3, 0.7, 0, 0, 0]$, defined for you as `Q_START`. [1]

Gradescope: Using `test_const_torque`, what happens if you apply a constant input torque of 0 to each joint?

A. The arm stays at the start configuration: with no input, nothing moves it.
B. The arm drifts slowly away from the start configuration as numerical integration errors accumulate.
C. The arm swings down back and forth under gravity like a multi-link pendulum.
D. The arm collapses straight down and hangs motionless within a second.

(b) **Constant torque** Start the robot at configuration `Q_START`. [1]

Gradescope: What happens if you apply a constant input torque of 20 N·m to each joint?

A. The arm holds its start configuration: the applied torque balances gravity.
B. Every joint spins around at a constant rate, forever.
C. The arm behaves exactly as it does with zero torque: 1 N·m is too small to matter against gravity.
D. The joints wind up and jam against their position limits.

What would happen on a real robot?

(c) **A proportional controller** Now we will add a proportional controller to the iiwa. [5]

One problem is that the Drake state for this simulation is 14-dimensional (positions and velocities). Use the `Observer` you wrote for 8b to map the 14D state into a 7D position observation which is needed as input to your `PController`.

Gradescope: Use the controller, with a gain of 100, to try to maintain the original configuration for 10 seconds. Which best describes the behavior?

A. The arm stably holds the start configuration.
B. The arm oscillates about the start configuration with growing amplitude.
C. The arm oscillates with decreasing amplitude but well below the original configuration.

D. The arm falls exactly as it does with zero input torque.

(d) **A PD controller** Later in the class, we will study more sophisticated controllers in detail. But, for now, try out the PD controller that is defined for you in `dtsystems.py`. How does it affect the system's behavior? [5]

It estimates velocity by differencing successive position measurements, so its `dt` argument is the rate at which the controller itself runs; make sure you pass one that suits the simulation rather than relying on the default.

Gradescope: With a controller gain of 100 and a damping gain of 30, drive the arm from the start configuration through the step on joints 1, 2 and 4 that `test3` commands. How long does it take for the arm to come to rest? (Use `plot_log`, and/or watch the meshcat recording.)

(e) **IIWA Trajectory** Adapt your trajectory follower for the IIWA. Use it to track `SQUARE_IIWA` (initialize the position to be the first waypoint), a sequence of joint configurations given in `iiwa_1.py` whose end-effector positions are the corners of a 0.4 m square in a vertical plane in front of the robot, with the end-effector frame held axis-aligned (we found them with inverse kinematics, a tool we will study later in the term), so that the end effector draws the square. [5]

Use proportional gain of 10000 and derivative gain of 3000, and epsilon of 0.01. Play the simulation back in your browser and upload a screen recording of it to Gradescope (an mp4 file, much less than 500MB). Gradescope should indicate "Submitted"; the points for this problem will be assigned when the staff grades it.

2.1 Additional problems for 6.4212

For each question below submit a PDF which has a few sentences and at least 1 plot. Each answer should not be more than 1 page long (including plots).

Problem 10 Step size 6.4212 only [10]

There are two step sizes which impact the behavior we observe in the simulations for this pset: the plant's integration time step, and the controller's update period. How does changing the value of one or both of these step sizes impact the simulation?

Some ideas (feel free to explore this in other directions, and you don't need to answer all these questions):

1. How do these two step sizes impact/relate to each other?
2. Do you observer particular failure modes at certain values and what causes them?
3. Vary the plant's time step and use the smallest time step as the ground truth and compute error as the time step increases.
4. How do these step sizes impact wall-clock-time for computing the simulation?

Problem 11 Damping 6.4212 only [10]

Experiment with the damping parameter in the PD controller. How does it affect the performance of the robot at following a trajectory?

Some ideas (feel free to explore this in other directions, and you don't need to answer all these questions):

1. Command a step change to a single joint and sweep the damping gain in the PD controller.
2. Identify responses that are under-damped, over-damped and critically damped.
3. How does overshoot and settling time vary with the gain?
4. Does the behavior depend on the arms configuration? Why, or why not?
5. Rerun the square-drawing task at several damping gains and plot the tracking error along the square and the time the arm takes to complete it. In what ways does more damping help or hurt?

3 What you should understand about the code you just produced

- How to construct and add a new DTSystem module — not at the code API level, but at the content level: what are the states, update and output functions? What would the resulting diagram look like?
- You should be able to draw the system diagram for any system you build.
- Debugging skills: be able to add loggers to the diagram and plot their output, print the block diagram of systems, search and read the Drake documentation.
- Why `dt` needs to be an argument to our physics simulation and the PD controller but not the simple proportional controller or the waypoint follower.
- Why the controller gain matters.
- What role the declaration of the inertia of objects in an SDF file plays.
- What role the declaration of joint damping in an SDF/URDF file plays.
- How changing epsilon in the simple trajectory follower would change the path the robot follows. How does that interact with the controller gain?